

Fuzzing Harness Development for SumatraPDF Reader

Team Members:

1. G. Venkatanatha – CH.SC.U4CYS24016
2. N. Adarsh Krishna – CH.SC.U4CYS24038
3. P. Siddhesh – CH.SC.U4CYS24042

Abstract:

This project presents the design and implementation of an automated fuzzing harness for SumatraPDF, a popular PDF reader. The aim was to evaluate the robustness and reliability of SumatraPDF by generating mutated PDF inputs through random mutation techniques and systematically monitoring the application for crashes or parser errors. The developed pipeline incorporates techniques such as bit flipping, magic number insertion, and byte deletion to create malformed files, which are automatically executed in SumatraPDF. Application behavior—including exit codes and error dialogs—is monitored, with all results being logged and archived in an SQLite database. Despite extensive fuzz testing, no critical crashes or memory violations were discovered, demonstrating the tested version's robustness. The project also includes a user-friendly server interface and comprehensive data reporting. This work highlights the practical value of fuzz testing for software reliability assessment.

Introduction:

Fuzz testing is a technique used to discover vulnerabilities and robustness issues in software applications by generating and executing mutated inputs. The aim of this project was to build and evaluate an automated PDF fuzzing pipeline (SumatraPDF as the target). The pipeline detects and logs application errors, unhandled crashes, and parser failures for a series of mutated PDF inputs.

Overview:

Programming languages used: C++ (including minor use of Windows API)

Database : sqlite3

Fuzzer: Generates mutated versions of PDF file inputs using strategies such as byte flipping, magic number insertion, and byte deletion.

Server: Runs SumatraPDF as the target application, automatically opening each mutated input and reporting the exit code to the fuzzer.

The system workflow operates as follows:

1. Select seed PDF files as base inputs.
2. Apply random mutations to generate new test files.
3. Save each mutated PDF as a temporary file.
4. Launch SumatraPDF to open the mutant file and monitor its behaviour.
5. Record and archive any errors, crashes, or parser failures.

Fuzzing Methodology:

1. Mutation Techniques: BitFlip, ByteFlip, MagicNumber insertion, and DeleteBytes.
2. Seeds: Several valid and minimal PDF files were used.
3. Configuration: The system was typically run for 10 - 1,000 iterations per seed, with a mutation count of 10-20 per input.
4. Crash/Error Detection: Non-zero exit codes, including parser errors ("Error Loading") and system exceptions, were detected and logged.

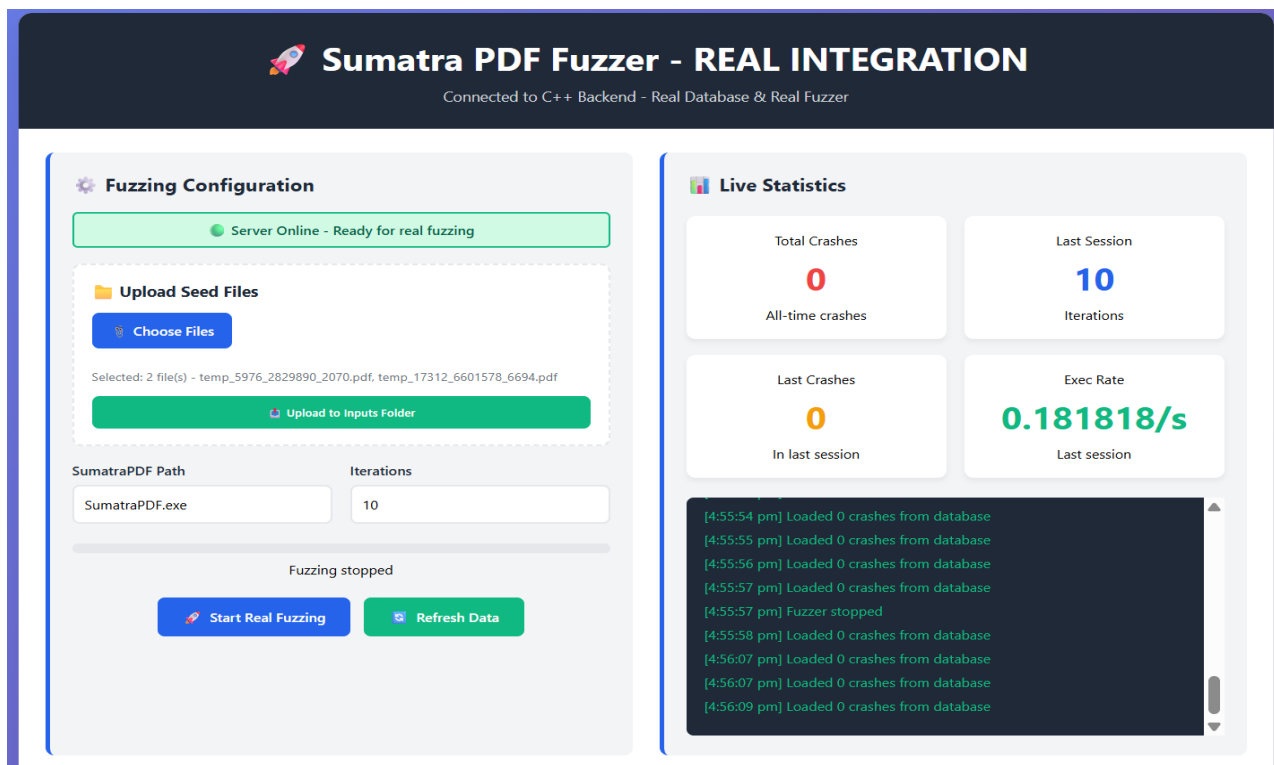
Execution:

Each fuzzing iteration follows this sequence:

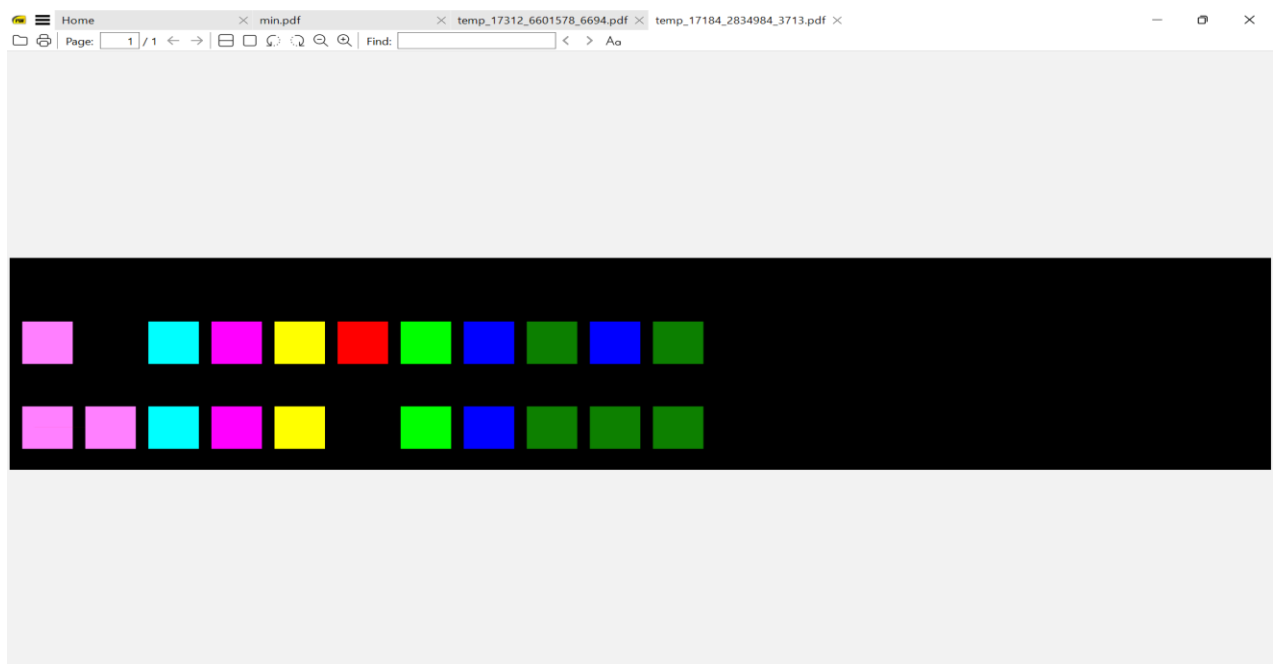
1. A seed PDF file is selected and mutated.
2. The mutant input is saved to disk as the temporary file.
3. SumatraPDF is executed to open the mutant.
4. The application exit code is checked.
5. If a crash or parser error is detected, the mutant input is moved to the outputs\crashes folder and a record is saved in the database.
6. And moreover, in the recent version of SumatraPDF if the temp file contains any malicious info, it will show the error in opening that file. Because that file is having error it cannot open that file which means that SumatraPDF is robust and secured in avoiding opening the malicious or corrupted files.

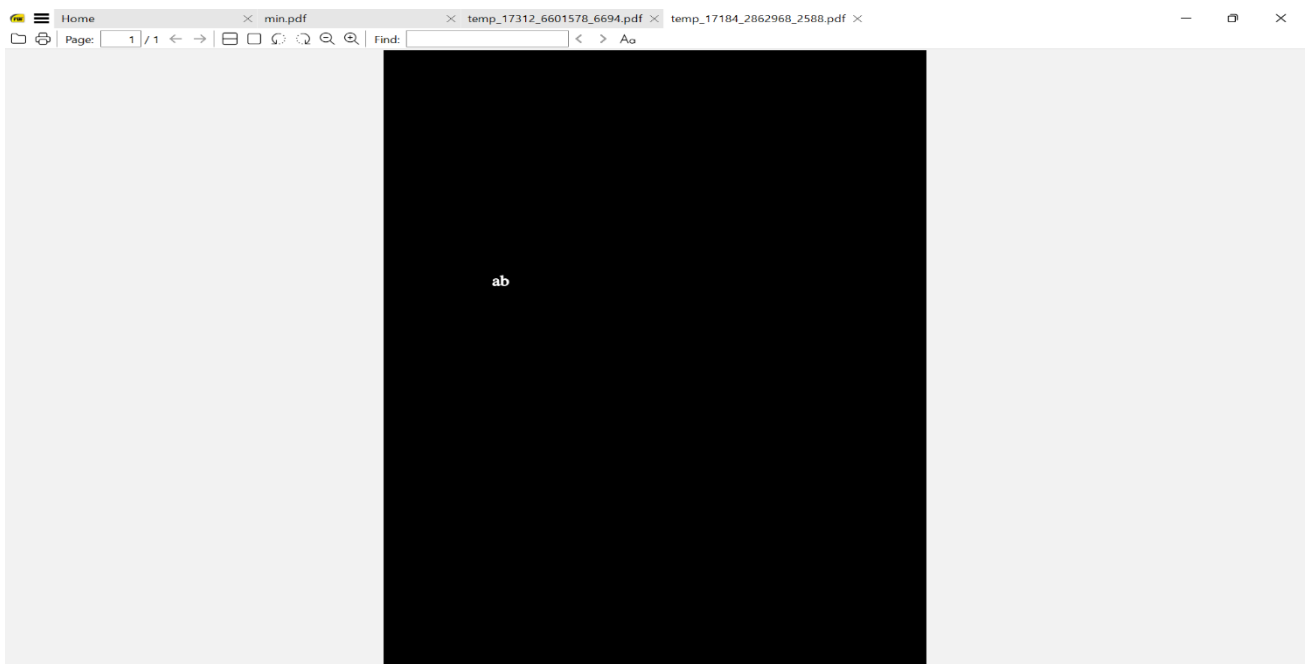
Step 1:

You will upload the files to server that you want to fuzz:



1. Here you can give number of iterations that you want to run for fuzzing the files from 0 -1000.
2. Live Statistics during the fuzzing session include: crash counts for the last session and all-time, iteration progress, execution rate, and for each session, a log of results and database archiving activity.
3. After choosing the files you can start the fuzzing.
4. During fuzzing this will open SumatraPDF reader according to the number of iterations you gave and go through each pdf or files that you have uploaded.





The workflow continues for the configured number of iterations .

Sumatra PDF Fuzzer - REAL INTEGRATION

Connected to C++ Backend - Real Database & Real Fuzzer

Fuzzing Configuration

Server Online - Ready for real fuzzing

Upload Seed Files

Choose Files

Selected: 2 file(s) - temp_5976_2829890_2070.pdf, temp_17312_6601578_6694.pdf

Upload to Inputs Folder

SumatraPDF Path

SumatraPDF.exe

Iterations

10

Fuzzing in progress... 100%

Stop Fuzzing

Refresh Data

Live Statistics

Total Crashes

0

All-time crashes

Last Session

10

Iterations

Last Crashes

0

In last session

Exec Rate

0.181818/s

Last session

[4:55:29 pm] Loaded 0 crashes from database

[4:55:30 pm] Loaded 0 crashes from database

[4:55:31 pm] Loaded 0 crashes from database

[4:55:32 pm] Loaded 0 crashes from database

[4:55:33 pm] Loaded 0 crashes from database

[4:55:34 pm] Loaded 0 crashes from database

[4:55:35 pm] Loaded 0 crashes from database

[4:55:36 pm] Loaded 0 crashes from database

[4:55:37 pm] Loaded 0 crashes from database

Real Crash Data from Database

If any crashes occurred in database, you can see them right below this fuzzing section:

Real Crash Data from Database

Recent Crashes

Database Info

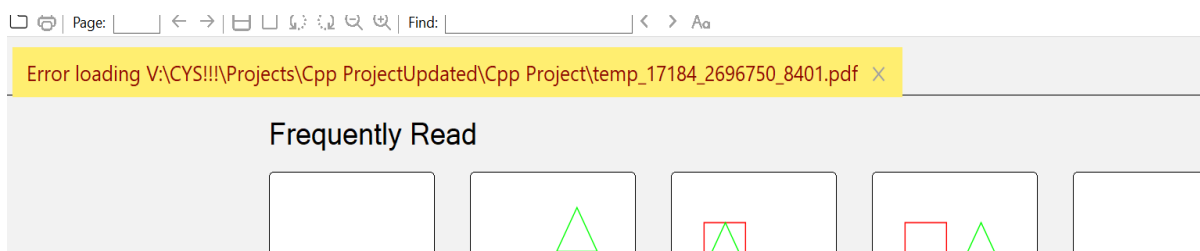
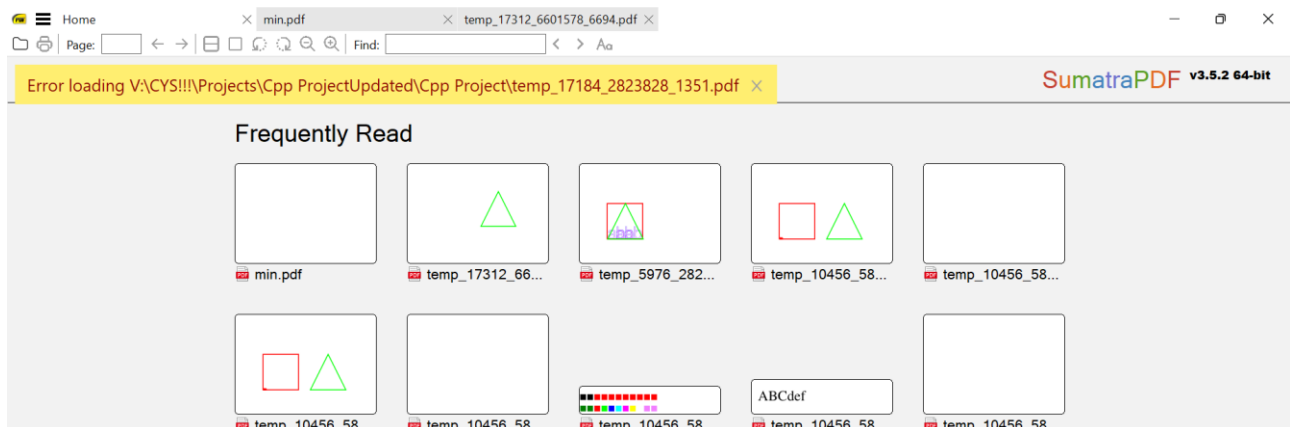
Console View

No crashes found in database

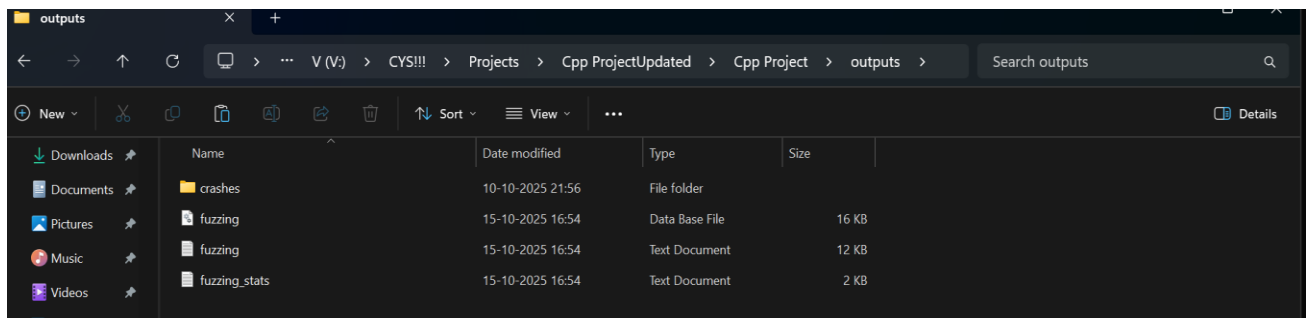
And as told that this fuzzer creates temp files (mutated files) after fuzzing the input seeds files will be stored in the project folder itself like this:

sqlite3	14-10-2025 13:21	Application	3,794 KB
SumatraPDF	14-10-2025 13:21	Application	15,689 KB
SumatraPDF-settings	15-10-2025 16:54	Text Document	304 KB
temp_5976_2829890_2070	14-10-2025 19:56	Microsoft Edge PDF ...	3 KB
temp_17312_6601578_6694	14-10-2025 20:59	Microsoft Edge PDF ...	3 KB

And next, we got these after fuzzing as a malicious report. So, as this is malicious and our SumatraPDF is so secured, it will not open these and displays like this:



After this it will store the session status and fuzzing status in a text file and in database too and stores these in the outputs folder.



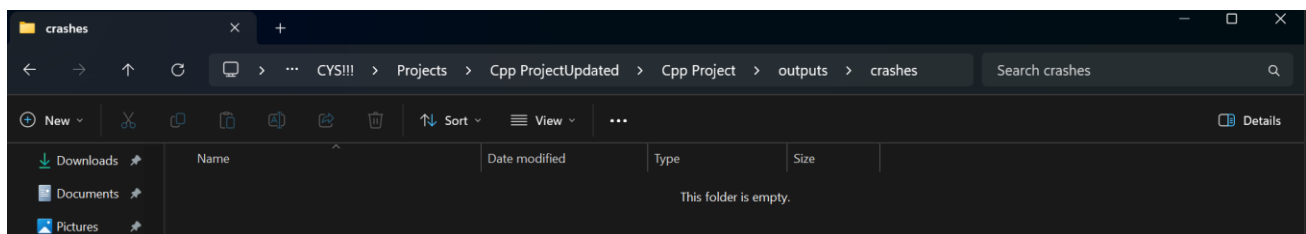
And fuzzing text file contains the following info:

```
[2025-10-15 16:53:53] [INFO] === Starting fuzzing session ===
[2025-10-15 16:53:53] [INFO] Target: SumatraPDF.exe
[2025-10-15 16:53:53] [INFO] Timeout: 5000ms
[2025-10-15 16:53:53] [INFO] Max iterations: 10
[2025-10-15 16:53:53] [INFO] Found 31 seed files
[2025-10-15 16:53:53] [INFO] Registered 5 mutators
[2025-10-15 16:53:53] [INFO] Loaded 30 valid seeds
[2025-10-15 16:53:53] [INFO] Starting fuzzing loop...
[2025-10-15 16:54:48] [INFO] === Fuzzing session complete ===
[2025-10-15 16:54:48] [INFO] Total iterations: 10
[2025-10-15 16:54:48] [INFO] Total crashes: 0
[2025-10-15 16:54:48] [INFO] Total timeouts: 10
[2025-10-15 16:54:48] [INFO] Total errors: 0
[2025-10-15 16:54:48] [INFO] Total time: 55 seconds
[2025-10-15 16:54:48] [INFO] Average rate: 0.18 exec/s
```

In fuzzing stats text file this content will be there:

```
SESSION|1760454840|1760454847|10|0|0|0|1.42857
SESSION|1760454996|1760455011|20|0|0|0|1.33333
SESSION|1760455251|1760455418|30|0|30|0|0.179641
SESSION|1760456340|1760456396|10|0|10|0|0.178571
SESSION|1760457561|1760457727|30|0|30|0|0.180723
SESSION|1760458150|1760458703|100|0|100|0|0.180832
SESSION|1760461584|1760462091|100|0|100|0|0.197239
SESSION|1760462212|1760462263|10|0|10|0|0.196078
SESSION|1760525583|1760526135|100|0|100|0|0.181159
SESSION|1760526498|1760526775|50|0|50|0|0.180505
SESSION|1760527069|1760527125|10|0|10|0|0.178571
SESSION|1760527433|1760527488|10|0|10|0|0.181818
```

In crashes for now we have no files because none of the crashes are occurred:



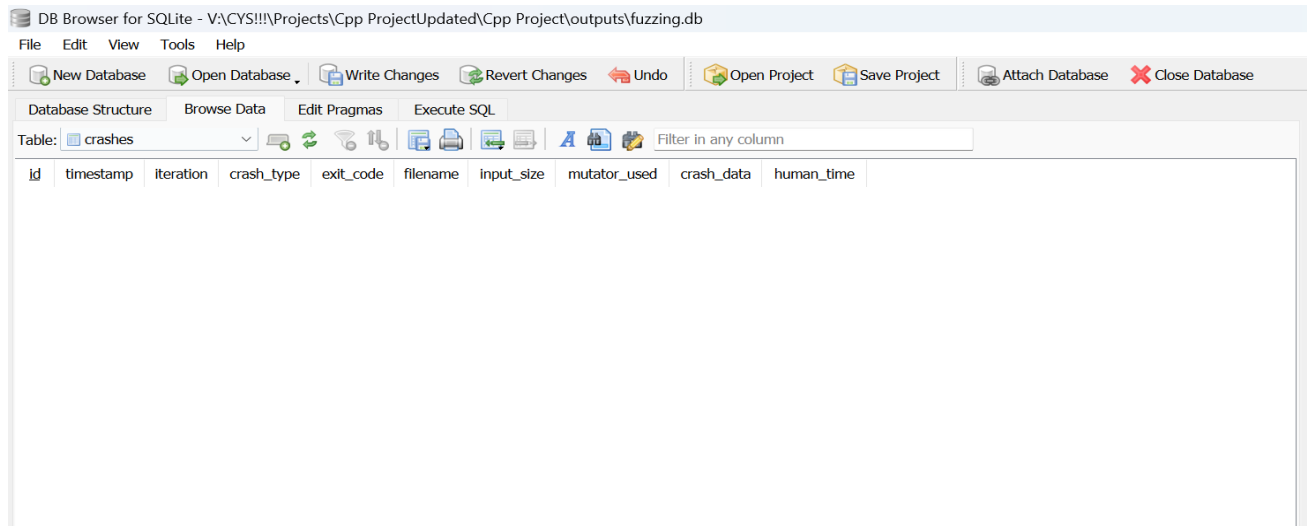
In Data base we can clearly see the sessions and crashes data

We used sqlite3 for creating database

Database:

We have created two tables in this for storing the data. One table is for storing the data about crashes and another one is for sessions that so far happened.

Crashes table:



DB Browser for SQLite - V:\CYS!!!\Projects\Cpp ProjectUpdated\Cpp Project\outputs\fuzzing.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

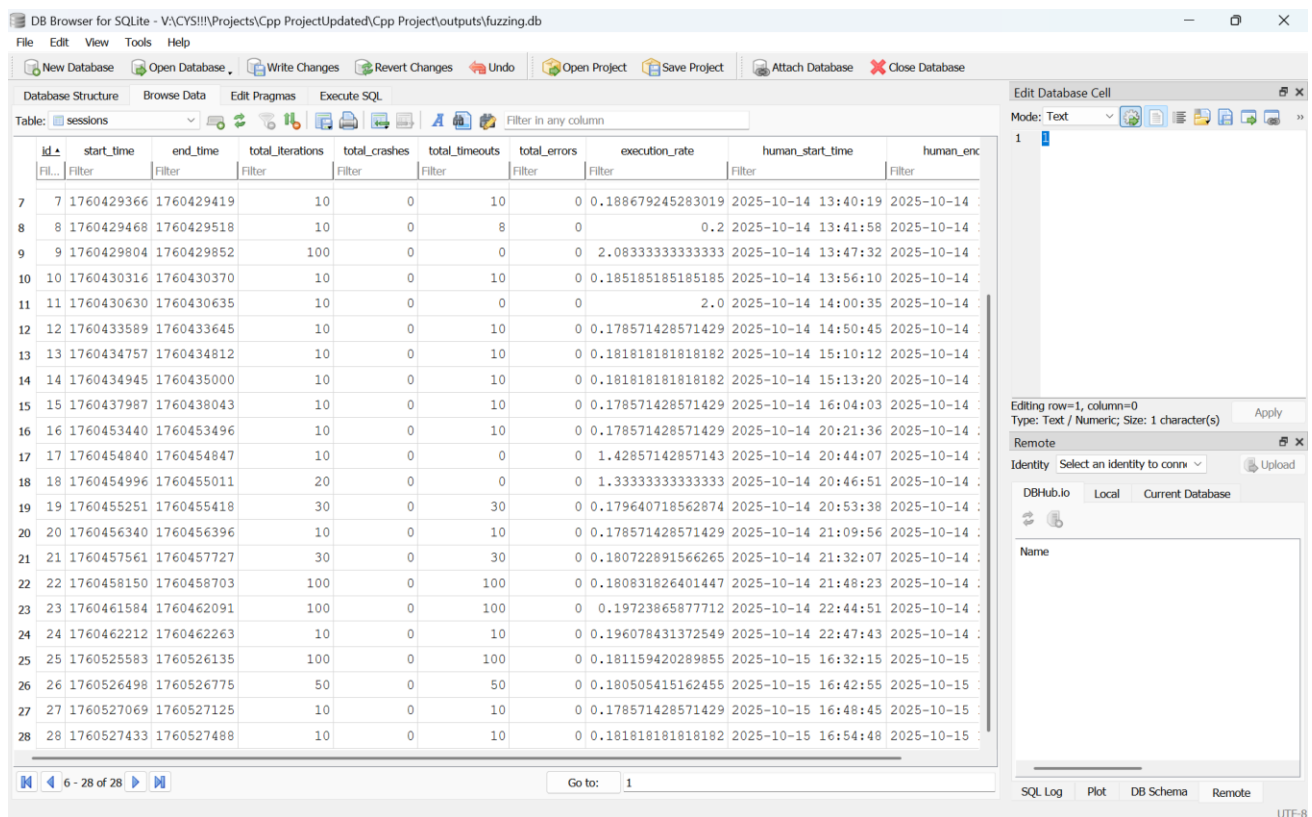
Database Structure Browse Data Edit Pragmas Execute SQL

Table: crashes Filter in any column

id	timestamp	iteration	crash_type	exit_code	filename	input_size	mutator_used	crash_data	human_time
----	-----------	-----------	------------	-----------	----------	------------	--------------	------------	------------

So, as we can see this table stores id, unix time, at what iteration we got crash, the type of crash, exit code, name of the file that has been crashed, input size, in which mutator we got crash, data that crashed and human time.

Sessions Table:



DB Browser for SQLite - V:\CYS!!!\Projects\Cpp ProjectUpdated\Cpp Project\outputs\fuzzing.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: sessions Filter in any column

id	start_time	end_time	total_iterations	total_crashes	total_timeouts	total_errors	execution_rate	human_start_time	human_end_time
7	1760429366	1760429419	10	0	10	0	0.188679245283019	2025-10-14 13:40:19	2025-10-14
8	1760429468	1760429518	10	0	8	0	0.2	2025-10-14 13:41:58	2025-10-14
9	1760429804	1760429852	100	0	0	0	2.08333333333333	2025-10-14 13:47:32	2025-10-14
10	1760430316	1760430370	10	0	10	0	0.185185185185185	2025-10-14 13:56:10	2025-10-14
11	1760430630	1760430635	10	0	0	0	2.0	2025-10-14 14:00:35	2025-10-14
12	1760433589	1760433645	10	0	10	0	0.178571428571429	2025-10-14 14:50:45	2025-10-14
13	1760434757	1760434812	10	0	10	0	0.181818181818182	2025-10-14 15:10:12	2025-10-14
14	1760434945	1760435000	10	0	10	0	0.181818181818182	2025-10-14 15:13:20	2025-10-14
15	1760437987	1760438043	10	0	10	0	0.178571428571429	2025-10-14 16:04:03	2025-10-14
16	1760453440	1760453496	10	0	10	0	0.178571428571429	2025-10-14 20:21:36	2025-10-14
17	1760454840	1760454847	10	0	0	0	1.42857142857143	2025-10-14 20:44:07	2025-10-14
18	1760454996	1760455011	20	0	0	0	1.33333333333333	2025-10-14 20:46:51	2025-10-14
19	1760455251	1760455418	30	0	30	0	0.179640718562874	2025-10-14 20:53:38	2025-10-14
20	1760456340	1760456396	10	0	10	0	0.178571428571429	2025-10-14 21:09:56	2025-10-14
21	1760457561	1760457727	30	0	30	0	0.180722891566265	2025-10-14 21:32:07	2025-10-14
22	1760458150	1760458703	100	0	100	0	0.180831826401447	2025-10-14 21:48:23	2025-10-14
23	1760461584	1760462091	100	0	100	0	0.19723865877712	2025-10-14 22:44:51	2025-10-14
24	1760462212	1760462263	10	0	10	0	0.196078431372549	2025-10-14 22:47:43	2025-10-14
25	1760525583	1760526135	100	0	100	0	0.181159420289855	2025-10-15 16:32:15	2025-10-15
26	1760526498	1760526775	50	0	50	0	0.180505415162455	2025-10-15 16:42:55	2025-10-15
27	1760527069	1760527125	10	0	10	0	0.178571428571429	2025-10-15 16:48:45	2025-10-15
28	1760527433	1760527488	10	0	10	0	0.181818181818182	2025-10-15 16:54:48	2025-10-15

6 - 28 of 28

Go to: 1

Edit Database Cell

Mode: Text

1

Editing row=1, column=0

Type: Text / Numeric; Size: 1 character(s)

Apply

Remote Select an identity to connect

Identity

DBHub.io Local Current Database

Upload

Name

SQL Log Plot DB Schema Remote

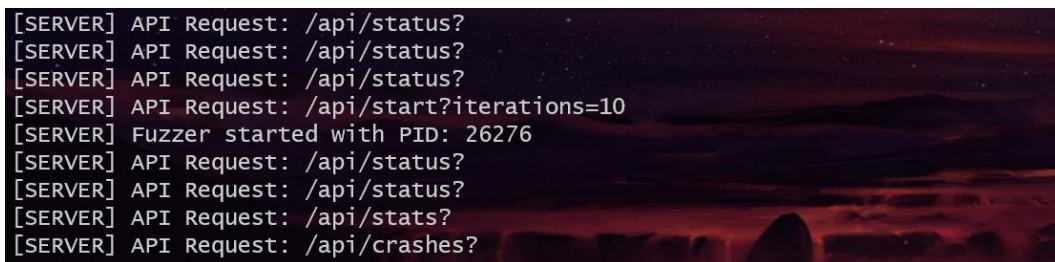
UTF-8

And the sessions table stores id, unix start time and end time, how many iterations, total number of crashes, total timeouts, errors if any, execution rate, human start time and end time.

Most archived events were robust parser failures handled gracefully by SumatraPDF. No actual memory violation crashes were observed; this highlights the stability of the tested SumatraPDF release. Crash database and file archiving pipeline worked end-to-end, demonstrating all features effectively.

Server:

We have added GUI using the server, connecting it to localhost:



```
[SERVER] API Request: /api/status?  
[SERVER] API Request: /api/status?  
[SERVER] API Request: /api/status?  
[SERVER] API Request: /api/start?iterations=10  
[SERVER] Fuzzer started with PID: 26276  
[SERVER] API Request: /api/status?  
[SERVER] API Request: /api/status?  
[SERVER] API Request: /api/status?  
[SERVER] API Request: /api/status?  
[SERVER] API Request: /api/status?
```

The above screenshot shows console logs from the server component during an automated fuzzing session. The [SERVER] API Request messages indicate the fuzzer is communicating with the server over HTTP endpoints, periodically polling status and retrieving information about the ongoing fuzzing campaign.

This server-based architecture enables monitoring and remote control of the fuzzing process via HTTP API requests.

Conclusion:

The automated fuzzing pipeline successfully generated, executed, and archived malformed PDF files that SumatraPDF was unable to open. While no security-critical crashes were discovered, the experiment demonstrates the real-world utility of fuzz testing for defensive programming and software robustness assessment. The output reporting, crash archiving, and data logging components all functioned as designed.

References and Resources:

1. Microsoft Docs: CreateProcessA API
<https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessa>
2. SumatraPDF Project
<https://www.sumatrapdfreader.org/>

3. MinGW (Minimalist GNU for Windows)

<http://www.mingw.org/>

4. SumatraPDF Vulnerabilities List

https://www.cvedetails.com/product/62114/Sumatrapdfreader-Sumatrapdf.html?vendor_id=21018